SFS.Labs

SELECTED CLIENT ENGAGEMENT / UNISWAP V4

From security findings to verifiable code changes.

POINCARÉ · An independent scoped Solidity security review and subsequent remediation follow-up.

A project-specific assessment of a custom-curve hook and detector: evidence-driven review, actionable findings, developer remediation, and a second source-level assessment of the resulting changes.

7	2	8	2
Solidity source files in scope	Low-severity findings	Informational observations	Independent review deliveries

Mathias Steinbach · Independent Solidity Security Reviewer

26 OCT 2026

What was reviewed, and why.

POINCARÉ is a Uniswap v4 custom-curve hook that uses a change-point detector to inform a bounded directional spread. The assignment was to assess seven first-party Solidity files at one pinned Git commit, not to certify the protocol or its economics.

Client context

Core security questions

- Intra-block detector manipulation
- Round-trip value extraction
- First-depositor/share inflation
- Quote-to-execution fidelity
- Fee and spread bounds

Method

Independent verification

- Manual call-path and trust-boundary review
- Foundry test-suite reproduction
- Slither and Aderyn triage
- Echidna property checks
- Bounded off-chain integer modeling

EVIDENCE AT ORIGINAL COMMIT · 78D75E24

222/222
Foundry tests independently run at original pin

10,001
Echidna calls · three non-exploit properties passed

50,000
Off-chain arithmetic-model trials in original assessment

REVIEW LIFECYCLE

01 · PIN
Exact source revision

›

02 · REVIEW
Inspect, test, triage

›

03 · REMEDIATE
Client changes & regression tests

›

04 · FOLLOW UP
Review fixed revision

Boundary: These checks improve confidence in the exercised code paths, but do not establish comprehensive protection against MEV, production deployment risks, third-party dependencies, or undiscovered vulnerabilities.

Two actionable defects. Eight documented observations.

The review did not confirm a Critical, High or Medium vulnerability within its defined methods and scope. Its strongest contribution was concrete guidance tied to exact contract behavior.

LOW · L-1

First-pool initialization could bind an unintended PoolKey.

The inherited initializer accepted whichever valid pool key reached a newly deployed hook first. Deployment and initialization were separate transactions, leaving room for a competing initialization before the intended one.

Recommended: Pin the intended PoolKey at construction or perform actually atomic initialization; verify the bound key before publishing addresses.

LOW · L-2

Detector price quantization distorted small raw-unit ratios.

WAD-scaled integer mids could collapse price movements into coarse steps on decimal-mismatched or small-ratio pairs, altering the signal that drives bounded fees and spread.

Recommended: Increase internal price resolution or explicitly constrain supported pairs; cover zero-price recovery and precision at relevant extremes.

EIGHT INFORMATIONAL NOTES · N-1 TO N-8

Control & accounting

- Trend relabeling under binding rate limits
- Nonbinding deployed rate limit
- Liquidity-lock nesting behavior
- Dust burns yielding no assets

Configuration & economics

- Parameter bounds / extreme arithmetic
- Lens arithmetic and stale comments
- Supported-token assumptions
- JIT liquidity and cross-block MEV

Severity discipline: A finding is only a confirmed issue to the extent supported by its method and evidence. The initial report deliberately did not claim an exploit proof, a formal proof, or complete economic safety.

Findings converted into implementation.

The project maintainer responded with a dedicated branch, tests, and documentation. The independent follow-up traced the specific changes against the original acceptance criteria.

9	26	240
Commits ahead of the reviewed baseline	Changed files in the GitHub comparison	Foundry tests reported passing by developer · not independently rerun

L-1	**Wrong-key binding prevented** Constructor pins `expectedPoolId`; `_beforeInitialize` checks it. Both deployment paths verify the bound pool.	CODE PRESENT
L-2	**Precision increased to 1e36** Detector uses high-resolution `marginalPrice`; historical WAD getter/event convert the value back.	CODE PRESENT
N-4	**Zero-payout dust burns rejected** Liquidity removal reverts rather than burning shares for no assets.	CODE PRESENT
N-5	**Extreme parameter logic hardened** Overflow-safe rate limiting and constructor validation of `dFloor`.	CODE PRESENT
N-6	**Stale reserve comment corrected** Other edge cases were documented rather than claimed fully eliminated.	PARTIAL

NOT EVERY OBSERVATION BECAME A CODE CHANGE

> N-1/N-2 were evaluated against the pinned configuration; N-3 accepted as a maintenance risk; N-7 documented as a token policy. **N-8 remained open:** adversarial liquidity timing and multi-block MEV economics were not proven safe.

Branch: Prakhar-30/POINCARE · sfs_labs_audit_work

A stronger evidence trail. Not a blanket sign-off.

The second review connected original findings to actual implementation changes and surfaced three unresolved matters before treating remediation as technically closed.

Verified / source

Confirmed

- Remediation branch & nine commits
- Changes addressing L-1, L-2, N-4, N-5
- New Foundry regression test source
- Two sets of 200,000 off-chain arithmetic checks
- Documented disposition of all ten items

Not independently established

Remaining limits

- Client-reported 240/240 fresh Foundry result
- Canonical-router fork reproduction
- Production bytecode equivalence
- Comprehensive dependencies / MEV economics
- Deployment authorization or certification

FOLLOW-UP OBSERVATIONS

01 **Correct-key initialization race** — OPEN

Wrong-key prevention does not stop a third party from initializing the intended key first, potentially with an unexpected initial price.

02 **Preview interface compatibility** — OPEN

The public `previewDetector()` struct changed from WAD-scale `priceWad` to 1e36-scale `mid`; consumers need verification.

03 **Zero-mid recovery coverage** — OPEN

The regression demonstrates initial zero-mid recovery, but not an established baseline followed by a zero-mid interval.

> **Outcome:** Independent security observations were adopted in the client's codebase and reinforced with regression test code. The follow-up identified residual integration/deployment considerations rather than claiming all risks resolved.

PROJECT & REVIEWER

Reviewer: **Mathias Steinbach / SFS.Labs** · Independent contributor, not a security-audit firm.
Source: github.com/Prakhar-30/POINCARE · Reviewer: github.com/SFS-Labs
This case study references the review and public code changes; it does not reproduce the confidential report or evidence archive.